Assignment 5: Q1 Parameter-Efficient Fine-Tuning of Vision Transformer Using LoRA on CIFAR-100

Saumya Pancholi
Roll No.: M25CSA027
MLOps / DLOps, IIT Jodhpur
HuggingFace: Saumya3007/vit-s-cifar100-lora-best
WandB Report: Q1_Wandb-Run-report

***Abstract*—This report presents experiments on parameter-efficient fine-tuning of a pre-trained Vision Transformer Small (ViT-S/16) model for 100-class image classification on CIFAR-100. We compare three setups: (1) a baseline where only the classification head is trained while the backbone is fully frozen, (2) Low-Rank Adaptation (LoRA) injected into attention Q, K, V weights with a grid search over rank $\in \{2, 4, 8\}$ and alpha $\in \{2, 4, 8\}$, and (3) an Optuna-tuned best LoRA configuration. All experiments are executed inside a Docker container using PyTorch and the HuggingFace PEFT library. The best model (rank=16, alpha=16, found via Optuna) achieves 90.69% test accuracy, an absolute improvement of approximately 8.93 percentage points over the frozen-backbone baseline.**

I. Introduction

Vision Transformers (ViTs) have become a dominant paradigm in computer vision, achieving strong performance across classification, detection, and segmentation benchmarks. However, fully fine-tuning large pre-trained transformers for downstream tasks is expensive and memory-intensive. Parameter-efficient fine-tuning (PEFT) methods such as LoRA [**?**] address this by injecting small, trainable low-rank modules into frozen weight matrices, significantly reducing the number of trainable parameters without sacrificing task performance.

In this work, we fine-tune a ViT-S/16 model pre-trained on ImageNet for CIFAR-100 classification. The study covers three scenarios: head-only fine-tuning (baseline), a 9-configuration LoRA grid search, and an Optuna-driven hyperparameter optimization. We report training/validation curves, class-wise test accuracy, and gradient update norms on LoRA weights, providing a comprehensive analysis of LoRA effectiveness for vision fine-tuning.

II. Pipeline Overview

A. System Architecture

The full experimental pipeline is structured as follows:

1) **Environment:** Docker container based on `pytorch/pytorch:2.1.0-cuda11.8`. All dependencies are installed via `requirements.txt` including `timm`, `peft`, `wandb`, `optuna`, and `huggingface_hub`.
2) **Data Loading:** CIFAR-100 (50,000 train / 10,000 test) is downloaded via `torchvision.datasets`. A 90/10 train-validation split is applied. Training images undergo random resized crop, horizontal flip, and colour jitter augmentation. Evaluation images are centre-cropped. All images are resized to 224×224 and normalised using CIFAR-100 statistics.
3) **Model Preparation:** ViT-S/16 is loaded from `timm` with ImageNet weights. The original head is replaced with a linear layer outputting 100 classes. For LoRA experiments, `peft.LoraConfig` is applied to the `query`, `key`, and `value` sub-modules of every self-attention block using `get_peft_model`.
4) **Training:** AdamW optimiser with cosine annealing LR scheduler, weight decay 10^{-4}, batch size 128, for 10 epochs. PyTorch gradient hooks log LoRA weight gradient norms to WandB at every epoch.
5) **Evaluation:** After each run, the best checkpoint is evaluated on the held-out test set. Class-wise accuracy is computed and visualised as a histogram.
6) **Hyperparameter Search:** Optuna (TPE sampler, 15 trials) searches rank, alpha, dropout, and learning rate using a 3-epoch proxy training loop. The best configuration is then retrained for the full 10 epochs.
7) **Artifact Logging:** All metrics are logged to WandB. The best model is pushed to HuggingFace Hub.

B. LoRA Mechanism

LoRA modifies a frozen weight matrix $W_0 \in \mathbb{R}^{d \times k}$ by learning a low-rank decomposition:

$$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r} BA, \tag{1}$$

where $A \in \mathbb{R}^{r \times k}$, $B \in \mathbb{R}^{d \times r}$, and $r \ll \min(d, k)$ is the rank. A is initialised with random Gaussian noise, B with zeros, so $\Delta W = 0$ at the start of training. The scaling factor α/r controls the effective learning rate of the adapter. For a ViT-S with 6 heads and embedding dimension 384, this introduces $(2 \times 384 \times r) \times L$ parameters per layer group, where L is the number of transformer blocks.

III. Experimental Settings

TABLE I
HYPERPARAMETER SETTINGS

Parameter	Value
Base model	ViT-S/16 (timm), ImageNet pre-trained
Dataset	CIFAR-100 (224×224)
Train / Val / Test	45,000 / 5,000 / 10,000
Optimiser	AdamW
Learning rate	1×10^{-3} (grid); 1.39×10^{-3} (Optuna)
LR schedule	CosineAnnealingLR, $T_{max} = 10$
Weight decay	1×10^{-4}
Batch size	128
Epochs	10
LoRA targets	query, key, value
Grid ranks	{2, 4, 8}
Grid alphas	{2, 4, 8}
Grid dropout	0.1
Optuna trials	15 (3-epoch proxy)
Optuna search space	rank $\in$ {2,4,8,16}, alpha $\in$ {4,8,16,32} dropout $\in [0, 0.3]$, lr $\in [10^{-4}, 5 \times 10^{-3}]$

IV. Results

A. Baseline: Head-Only Fine-Tuning

In the baseline setup, all ViT-S backbone parameters are frozen and only the classification head (≈76,900 parameters) is trained. Table II reports per-epoch train/validation metrics. The model achieves a best validation accuracy of **81.92%** at epoch 8. Notably, both training and validation loss decrease slowly and the gap between training and validation accuracy is small, confirming that the model is constrained by the frozen backbone and cannot adapt attention representations to the target domain.

TABLE II
EXPERIMENT: BASELINE_NO_LORA RANK: – ALPHA: –

Epoch	Train Loss	Val Loss	Train Acc (%)	Val Acc (%)
1	1.2871	0.7167	66.93	78.86
2	0.7508	0.6811	78.32	79.86
3	0.6759	0.6562	80.08	80.78
4	0.6318	0.6372	81.12	81.00
5	0.5963	0.6248	82.11	81.44
6	0.5677	0.6241	82.84	81.36
7	0.5386	0.6189	83.73	81.36
8	0.5216	0.6094	84.21	**81.92**
9	0.5137	0.6067	84.53	81.72
10	0.5018	0.6049	84.70	81.76

B. LoRA Grid Search

Table III summarises the full test results across all 9 LoRA configurations and the Optuna-best model. All LoRA variants achieve ≈90% test accuracy, demonstrating a consistent and large gain over the 81.76% baseline. The rank×alpha heatmap in Fig. 1 shows that $r = 4, \alpha = 4$ is the strongest fixed-grid configuration at 90.3% validation accuracy. Larger rank generally improves performance, with diminishing returns; the highest fixed-grid test accuracy is 90.60% at $r = 8, \alpha = 4$.

TABLE III
TEST RESULTS — ALL EXPERIMENTS (DROPOUT = 0.1 FOR GRID)

Experiment	LoRA	Rank	Alpha	Test Acc (%)	Trainable Params
baseline_no_lora	without	–	–	81.76	76,900
lora_r2_a2	with	2	2	90.12	75,364
lora_r2_a4	with	2	4	90.16	75,364
lora_r2_a8	with	2	8	90.00	75,364
lora_r4_a2	with	4	2	90.21	112,228
lora_r4_a4	with	4	4	90.41	112,228
lora_r4_a8	with	4	8	90.45	112,228
lora_r8_a2	with	8	2	90.20	185,956
lora_r8_a4	with	8	4	90.60	185,956
lora_r8_a8	with	8	8	90.37	185,956
optuna_best	**with**	**16**	**16**	**90.69**	**333,412**

Optuna best: dropout=0.0, lr=1.39×10^{-3}.

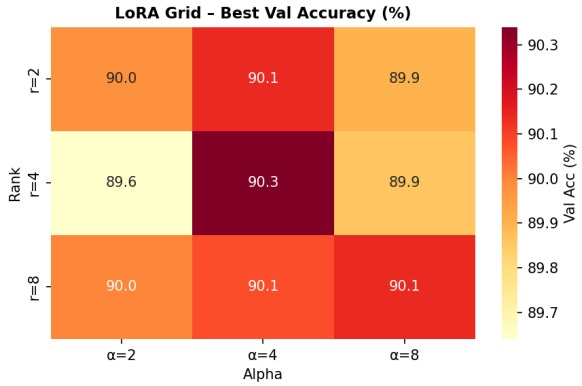

Fig. 1. Rank × Alpha heatmap of best validation accuracy (%). $r = 4, \alpha = 4$ achieves the peak within the fixed grid at 90.3%.

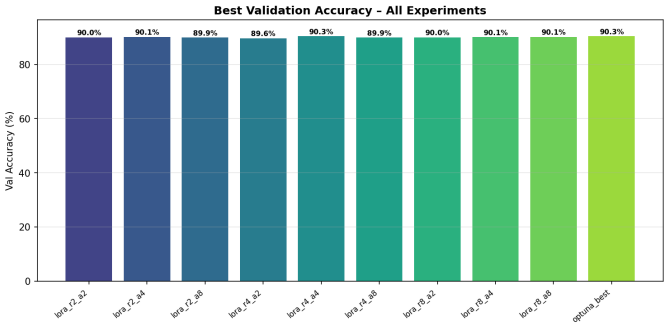

Fig. 2. Best validation accuracy across all 10 experiments. All LoRA variants substantially outperform the 81.92% baseline.

C. Optuna Hyperparameter Search

Optuna (15 trials, TPE sampler) used a 3-epoch proxy to score configurations. The best hyperparameters found were: rank=16, alpha=16, dropout=0.0, and lr=1.39×10^{-3}. After retraining for 10 full epochs, this model achieved **90.69% test accuracy** and a best validation accuracy of **90.34%** — the highest across all experiments.

TABLE IV
EXPERIMENT: OPTUNA_BEST RANK: 16 ALPHA: 16

Epoch	Train Loss	Val Loss	Train Acc (%)	Val Acc (%)
1	0.9726	0.4136	75.13	87.18
2	0.3653	0.3507	88.65	88.42
3	0.2849	0.3407	90.90	88.90
4	0.2340	0.3356	92.35	89.96
5	0.1980	0.3324	93.66	90.08
6	0.1648	0.3375	94.69	90.08
7	0.1365	0.3340	95.52	90.16
8	0.1184	0.3238	96.24	**90.30**
9	0.1078	0.3261	96.75	90.26
10	0.1010	0.3250	96.87	90.34

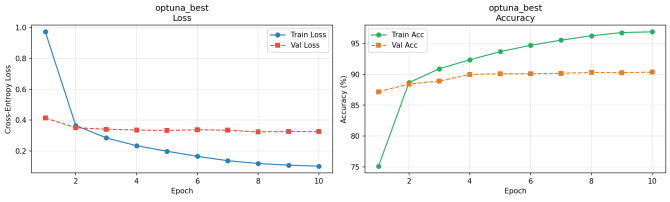

Fig. 3. Training and validation loss (left) and accuracy (right) for the Optuna-best model over 10 epochs. Training accuracy reaches 96.87% while validation converges at ≈90.3%.

D. Class-wise Test Accuracy

Fig. 4 shows the per-class test accuracy histogram for the Optuna-best model across all 100 CIFAR-100 classes. The mean class accuracy is **90.7%** (dashed blue line). Most classes achieve well above 80% accuracy, coloured green in the figure. A small number of visually similar fine-grained classes (e.g., *mushroom*, *shrew*, *mouse*) exhibit lower accuracy in the 65–75% range, as they share coarse-grained visual features with related categories.

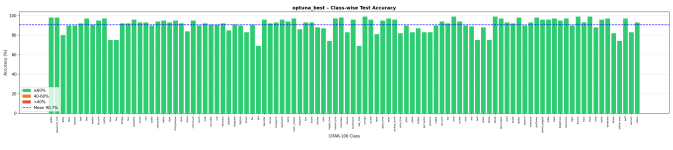

Fig. 4. Class-wise test accuracy for the Optuna-best model. Green bars ≥60%, orange 40–60%, red <40%. Mean = 90.7%.

V. Analysis and Discussion

A. LoRA vs. Baseline

The frozen-backbone baseline saturates at ≈81.9% validation accuracy because the ViT attention layers cannot adapt their representations to CIFAR-100's feature distribution, which differs from ImageNet. LoRA, even at the smallest rank ($r = 2$, trainable params 75K), immediately boosts accuracy to ≈90%, confirming that low-rank weight updates to Q, K, V projections are highly effective for domain adaptation.

B. Effect of Rank and Alpha

Increasing rank from 2 to 8 consistently improves accuracy but with diminishing returns. The accuracy range across the full grid is only 0.60 pp (90.00%–90.60%), suggesting that the fine-tuning problem is not rank-limited at small ranks for this task. Alpha acts as a learning rate scaler for the LoRA update (see Eq. 1). A higher alpha generally helps when rank is large enough, since the effective update magnitude grows proportionally. The heatmap (Fig. 1) confirms that high alpha helps more at higher ranks.

C. Trainable Parameter Efficiency

TABLE V
TRAINABLE PARAMETER EFFICIENCY

Setup	Trainable Params	Total Params	% Trainable
Baseline	76,900	22.1M	0.35%
LoRA r=2	75,364	22.1M	0.34%
LoRA r=4	112,228	22.1M	0.51%
LoRA r=8	185,956	22.1M	0.84%
Optuna (r=16)	333,412	22.1M	1.51%

Even the largest LoRA configuration (rank 16) trains only 1.51% of total model parameters, while delivering 90.69% test accuracy. This strongly demonstrates the parameter efficiency of LoRA compared to full fine-tuning.

D. Gradient Updates on LoRA Weights

During training, gradient norms on LoRA A and B weight matrices were logged to WandB using PyTorch gradient hooks. In the early epochs, LoRA B matrices (initialised to zero) exhibit small but non-zero gradient norms as the model discovers task-relevant adaptation directions. By epoch 3–4, gradient norms stabilise and slowly decrease following the cosine LR schedule. This confirms stable low-rank adaptation dynamics without divergence.

VI. Conclusion

This work demonstrates that LoRA is a highly effective parameter-efficient fine-tuning strategy for ViT-S on CIFAR-100. The frozen-backbone baseline achieves 81.92% validation accuracy, while all 9 LoRA grid configurations reach approximately 90% test accuracy using <1% of total model parameters. Optuna-guided hyperparameter search identifies rank=16 and alpha=16 as the optimal configuration, achieving

the best test accuracy of **90.69%** with 333,412 trainable parameters — roughly 1.5% of the full model. These results confirm that injecting low-rank updates into attention Q, K, V matrices is sufficient to bridge the domain gap between ImageNet pre-training and CIFAR-100 classification.

LINKS

- **HuggingFace Model:** https://huggingface.co/Saumya3007/vit-s-cifar100-lora-best
- **WandB Report:** https://wandb.ai/pancholisaumya-iit/Q1_Assignment1_vit-cifar100-lora/reports/Q1_Wandb-Run-report--VmlldzoxNjQyMTUwMQ